

Spell Checker Project

A spell checker is a program or function that checks for spelling errors in a body of text. In natural language processing (NLP), a spell checker can be used to pre-process text data in order to improve the accuracy of NLP models.

Finding Unique words in our corpus

To find unique words in a body of text in natural language processing (NLP), you can use a set data structure to store the words as they are encountered, and then check whether each word has already been added to the set before adding it. This will allow you to easily identify and count the number of unique words in the text.

1. Finding the Unique Words

```
[2]: with open('big.txt','r') as fd:
      lines = fd.readlines()
      words = []
      for line in lines :
          words += re.findall('\w+',line.lower())
      print(len(words))
      vocab = list(set(words))
      print(len(vocab))

<>:5: SyntaxWarning: invalid escape sequence '\w'
<>:5: SyntaxWarning: invalid escape sequence '\w'
C:\Users\v\AppData\Local\Temp\ipykernel_15496\3919749970.py:5: SyntaxWarning: invalid escape sequence '\w'
      words += re.findall('\w+',line.lower())
1115585
32198
```

In this way I have first taken all the unique words present in our corpus and the number of unique words present are 32198 .

Finding Probability Distribution

To find the probability distribution of words in a body of text in natural language processing (NLP), you can first create a frequency distribution of the words in the text.

This will give us the number of times each word occurs in the text.

Then, to convert the frequency distribution into a probability distribution, you can divide the frequency of each word by the total number of words in the text. This will give you the probability of each word occurring in the text .

2. Finding the Probability Distribution

```
[3]: word_probability = {}  
  
for word in tqdm(vocab):  
    word_probability[word] = float(words.count(word)/len(words))
```

100%|██████████| 32198/32198 [12:43<00:00, 42.16it/s]

Spell Checker Project

Text Preprocessing

Text preprocessing is the process of preparing text data for use in natural language processing (NLP) tasks. It involves a series of steps to clean, standardize, and normalize the text in order to make it more suitable for NLP algorithms to work with.

Splitting

In natural language processing (NLP), "splitting" refers to the process of dividing a body of text into smaller pieces or units. There are a few different ways that text can be split in NLP, depending on the specific goals and requirements of the task.

▼ **Splitting**

```
[5]: def split(word):  
    parts = []  
    for i in range(len(word)+1):  
        parts += [(word[ : i],word[i: ])]  
    return parts
```

`split('eshant')`

```
[('', 'eshant'),  
( 'e', 'shant'),  
( 'es', 'hant'),  
( 'esh', 'ant'),  
( 'esha', 'nt'),  
( 'eshan', 't'),  
( 'eshant', '')]
```

Deleting

In case the word is loave then we are supposed to delete the letter 'a' so that we get our original correct spelled word which is love.

Swapping

In case the word is lvoe then we are supposed to swap the letters 'v' amd 'o' to get our correct spelled word which is love.

Replacing

In case the word is lave then we are supposed to replace the letter 'k' with the letter 'o' to get our original correct spelled word which is love.

Inserting

In case the word is lve then we are supposed to insert the letter 'o' so that we get our correct spelled word which is love.

Spell Checker Project

Predicting the Correct word

So, basically here we will take a word and pass it through delete, swap, replace and insert and then we will get 4 different list of it. We will combine all the values which we are getting from the 4 different list and then finally find the unique words present in it. Finally we will get a combination of words and we will try to search if any word in the vocab(unique words present in the corpus) matches with the any of the word which we got in the combination after passing it through the delete, swap, replace and inserting process.

So, we find the answer in the two level .

5 Finding the Prediction (Level - 2)

5.1) Combining Possible Words

```
[19]: def spell_check_edit_2(word, count=5):
    output = []
    suggested_words = edit(word) # Level one edit
    for e1 in edit(word):
        suggested_words += edit(e1) # second Level Edit
    suggested_word = list(set(suggested_words))

    for wrd in suggested_word:
        if wrd in word_probability.keys():
            output.append([ wrd, word_probability[ wrd ] ])
    return list(pd.DataFrame(output, columns = ['word', 'prob']).sort_values(by='prob', ascending = False).head(count)['word'].values
    )
spell_check_edit_2('fameli')
```

```
[19]: ['family', 'namely', 'fame', 'camel', 'amelie']
```

4. Finding the Prediction(Level - 1)

4.1) Combining Possible Words

```
[10]: def edit(word):
    return list(set(insert(word)+delete(word) + swap(word) + replace(word)))
```

4.2) Predicting the Word

```
[15]: def spell_check_edit_1(word, count =5):
    output = []
    suggested_words = edit(word)

    for wrd in suggested_words:
        if wrd in word_probability.keys():
            output.append([ wrd, word_probability[ wrd ] ])
    return list(pd.DataFrame(output, columns=['word', 'prob']).sort_values(by = 'prob' , ascending = False).head(count)['word'].values)
```

```
[16]: spell_check_edit_1('famili')
```

```
[16]: ['family']
```

Github Link :- <https://github.com/Anshuman776/NLP-/blob/main/Spell%20Checking/Spell%20Checking.ipynb>